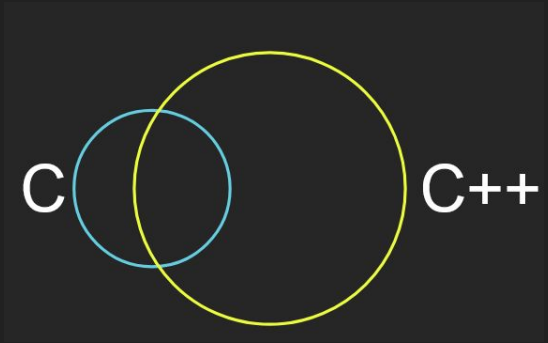
IT UNIVERSITY OF COPENHAGEN

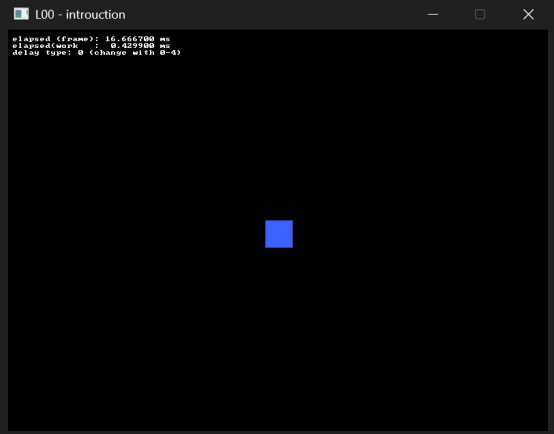
Game Programming 25

L01: Sprite Based Rendering

Chris Carvelli

Recap





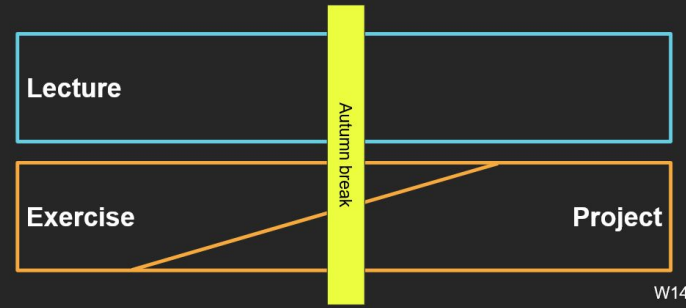
Game Loops

```c
// Simple loop
int main()
{
    init();

    while(!quit) // game loop
    {
        process_events();
        update();
        render();
    }

    return 0;
}
```

```c
// Fixed loop
int main()
{
    init();

    while(!quit) // game loop
    {
        float start = get_current_time();

        process_events();
        update();
        render();

        float end = get_current_time();
        float elapsed = end - start;

        sleep(MS_PER_FRAME - elapsed);
    }

    return 0;
}
```

```c
// Fluid loop
int main()
{
    init();
    float start = get_current_time();
    float delta = 0;

    while(!quit) // game loop
    {
        process_events();
        update(delta);
        render();

        float end = get_current_time();
        delta = end - start;
        start = end;
    }

    return 0;
}
```

Semester Structure



No lecture on 13/10

Exercise 00 review - types of delay

```c
// case 0: busy wait - very precise, but costly
SDL_Time walltime_busywait = walltime_work_end;
while(walltime_busywait - walltime_frame_beg < target_framerate_ns)
    SDL_GetCurrentTime(&walltime_busywait);
```

```c
// case 1: simple delay - too imprecise
// NOTE: `SDL_Delay` gets milliseconds, but our timer gives us nanoseconds! We need to convert it manually
SDL_Delay((target_framerate_ns - time_elapsed_work) / 1000000);
```

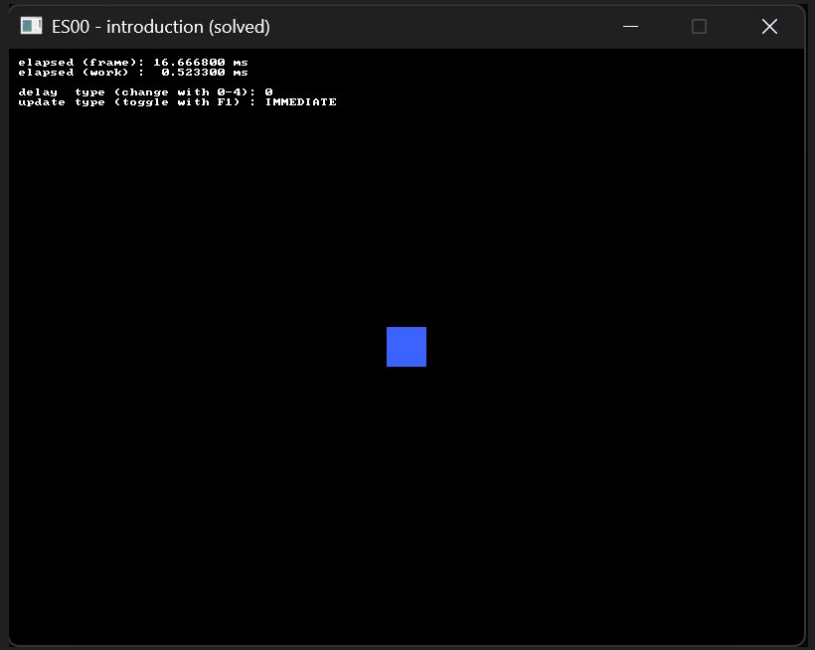
```c
// case 2: delay ns - also too imprecise
SDL_DelayNS(target_framerate_ns - time_elapsed_work);
```

```c
// case 3: delay precise - "It will attempt to wait as close to the requested time as possible, busy waiting if necessary, but could return later due to OS scheduling." (from SDL docs)
SDL_DelayPrecise(target_framerate_ns - time_elapsed_work);
```

```c
// case 4: custom delay - we use the sleeping delay with an arbitrary safety margin, then we busywait what's left
SDL_DelayNS(target_framerate_ns - time_elapsed_work - 1000000);
SDL_Time walltime_busywait = walltime_work_end;

while(walltime_busywait - walltime_frame_beg < target_framerate_ns)
    SDL_GetCurrentTime(&walltime_busywait);
```

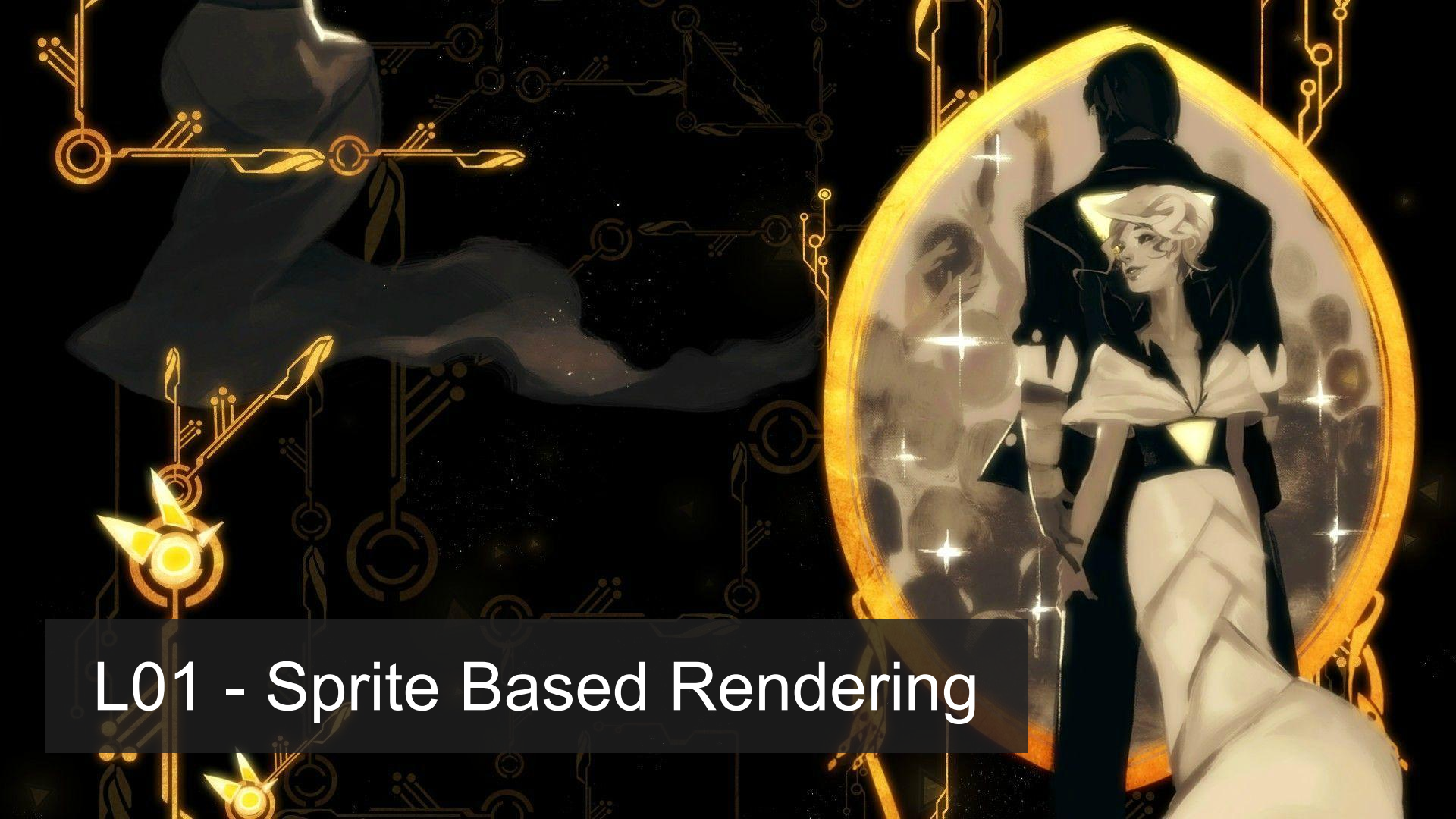
Exercise 00 review - player update

```
// raw update (poll event loop)
// game update tied to the whims of the hardware and OS

case SDL_EVENT_KEY_UP:
case SDL_EVENT_KEY_DOWN:
{
  if(event.key.key == SDLK_W) player_rect.y -= player_speed;
  if(event.key.key == SDLK_S) player_rect.y += player_speed;
  if(event.key.key == SDLK_A) player_rect.x -= player_speed;
  if(event.key.key == SDLK_D) player_rect.x += player_speed;
}
```



Exercise 00 review - player update

```
// insulated update (poll event loop)
case SDL_EVENT_KEY_UP:
case SDL_EVENT_KEY_DOWN:
{
  if(event.key.key == SDLK_W) btn_pressed_up    = event.key.down;
  if(event.key.key == SDLK_S) btn_pressed_down  = event.key.down;
  if(event.key.key == SDLK_A) btn_pressed_left  = event.key.down;
  if(event.key.key == SDLK_D) btn_pressed_right = event.key.down;
}

// update player position (later in the update loop)
if(btn_pressed_up)    player_rect.y -= player_speed;
if(btn_pressed_down)  player_rect.y += player_speed;
if(btn_pressed_left)  player_rect.x -= player_speed;
if(btn_pressed_right) player_rect.x += player_speed;
```



L01 - Sprite Based Rendering

Problem statement: How do we…

- …import digital images into games?
- …manipulate them so that they appear to be in a consistent and believable digital world?

- …pack them to be used in a smart and efficient way?
- …make them interact with each other?

Agenda

1. Vector math recap
2. 2D Graphics
3. C++ Tidbits

Exercise 00 Review

Agenda

1. Vector math recap
2. 2D Graphics
3. C++ Tidbits

Vector Math Basics

```
// given 2D vectors a, b
// and scalar f
```

sum	a + b	a.x + b.x, a.y + b.y
subtraction	a - b	a.x - b.x, a.y - b.y
element-wise multiplication	a * b	a.x * b.x, a.y * b.y
scale	a * f	a.x * f, a.y * f
length	length(a)	sqrt(a.x * a.x + a.y * a.y)
length_sq	length_sq(a)	a.x * a.x + a.y * a.y
dot product	dot(a, b)	a.x * b.x + a.y * b.y

Vector Math Basics

```
// given 2D vectors a, b
// and scalar f
```

sum	`a + b`	`a.x + b.x, a.y + b.y`
subtraction	`a - b`	`a.x - b.x, a.y - b.y`
element-wise multiplication	`a * b`	`a.x * b.x, a.y * b.y`
scale	`a * f`	`a.x * f, a.y * f`
length	`length(a)`	`sqrt(a.x * a.x + a.y * a.y)`
length_sq	`length_sq(a)`	`a.x * a.x + a.y * a.y`
dot product	`dot(a, b)`	`a.x * b.x + a.y * b.y`

doesn't really have a geometrical interpretation, but it's useful in many cases

Vector Math Basics

```
// given 2D vectors a, b
// and scalar f
```

sum	a + b	a.x + b.x, a.y + b.y
subtraction	a - b	a.x - b.x, a.y - b.y
element-wise multiplication	a * b	a.x * b.x, a.y * b.y
scale	a * f	a.x * f, a.y * f
length	length(a)	sqrt(a.x * a.x + a.y * a.y)
length_sq	length_sq(a)	a.x * a.x + a.y * a.y
dot product	dot(a, b)	a.x * b.x + a.y * b.y

square roots are expensive, so we should avoid them when we can
(ie, comparisons)

Dot Product Tests

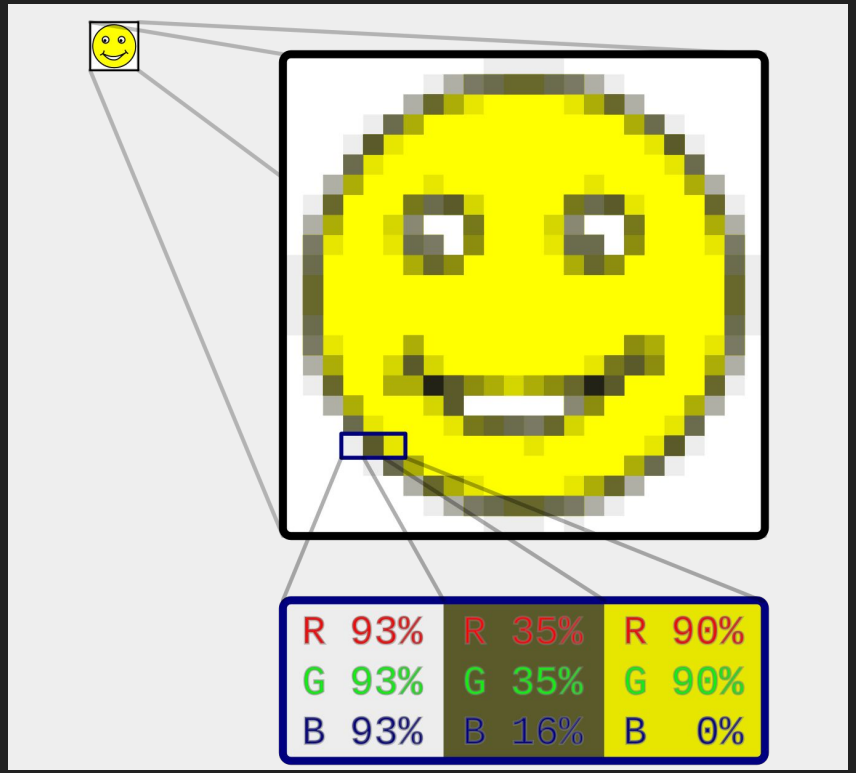
- Angle is $v \cdot w = \|v\|\|w\| cos\theta \rightarrow cos\theta = \frac{v \cdot w}{\|v\|\|w\|} \rightarrow \theta = arccos \frac{v \cdot w}{\|v\|\|w\|}$
- Magnitude as a dot product $\|v\|^2 = v \cdot v$
- Collinear: angle == 0 $v \cdot w = \|v\|\|w\|$
- Collinear but opposite: angle == 180 $v \cdot w = -(\|v\|\|w\|)$
- Perpendicular: angle == 90 $v \cdot w = 0$
- Same direction: angle < 90 $v \cdot w > 0$
- Opposite directions: angle > 90 $v \cdot w < 0$

Agenda

1. Vector math recap
2. 2D Graphics
3. Game Development 102

Raster graphics

- Image produced as an array (the raster) of
- picture elements (pixels) in the frame buffer

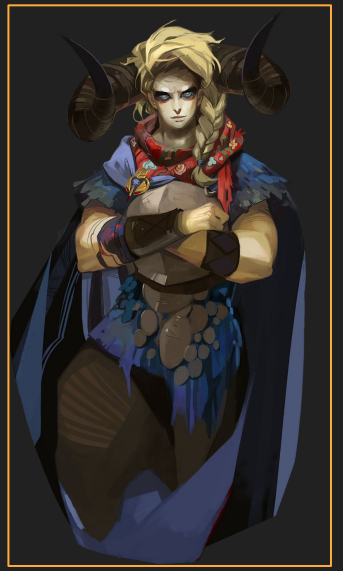


Pictures

image.png
(potentially compressed)

0	0	0	0	255	255	255	255	1	12
42	42	42	0	0	1	1	1	0	1

metadata
- width
- height
- num_channels
- ...



texture

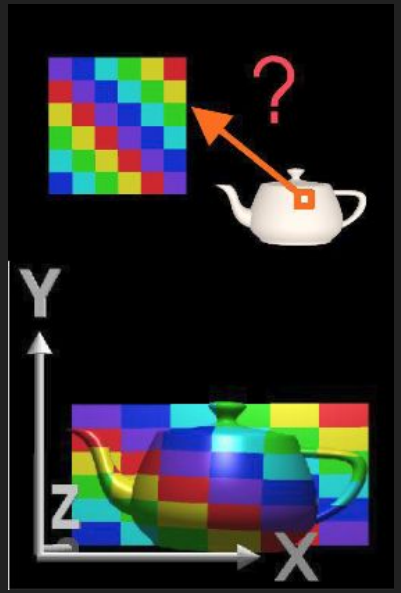
Images

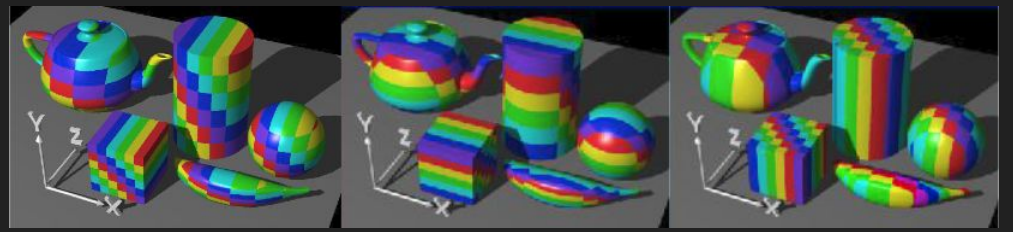
- Raster images can have any rectangular shape
- File formats:
  - Uncompressed: raw, bmp
  - Compressed: jpeg, png
- When loaded it is often stored uncompressed in main memory
- Hardware accelerated 2D graphics was a thing in the 80's

image.png

Textures

- Textures are images that are uploaded to the GPU.
- Textures are used by modern GPUs for both 2D and 3D graphics.
- Textures are used to store: colors, shadows, geometry data, etc.

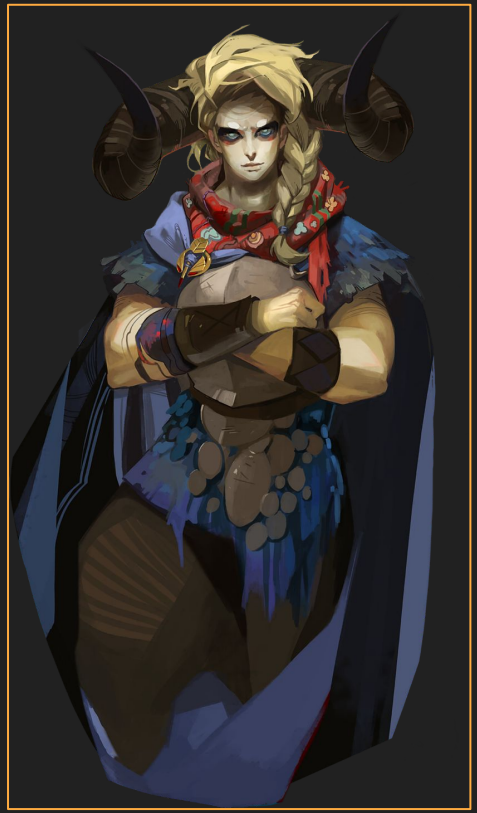




https://web.eecs.umich.edu/~sugih/courses/eecs487/lectures/24-TextureMapping.pdf

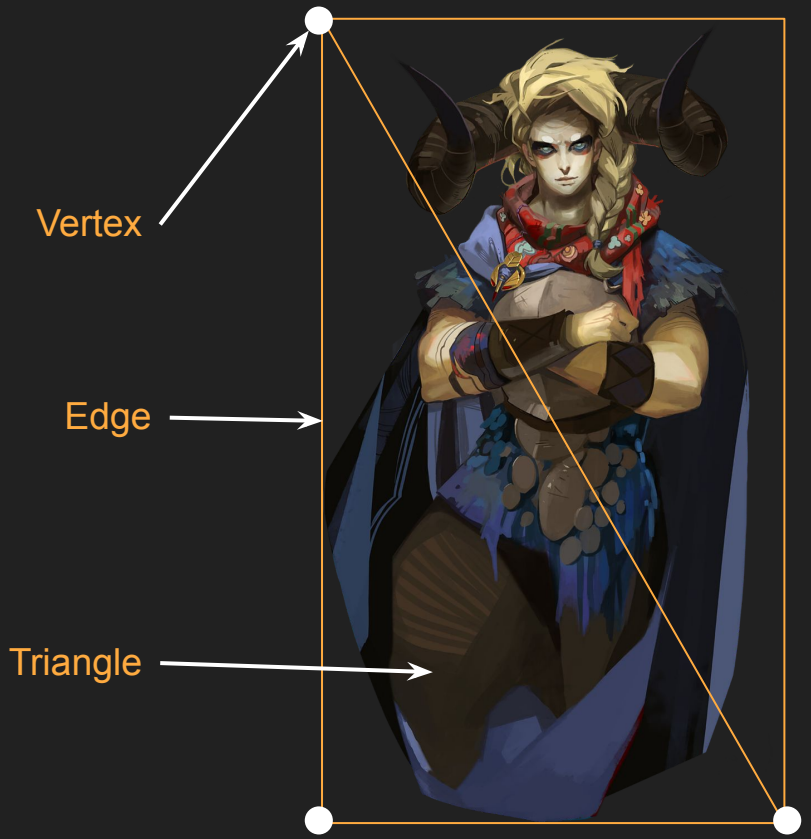
Texture rendering

- How can we render a texture in modern engines?
- Rendering engine these days treat 2D graphics as a special case of 3D.



Triangles

- When rendering graphics everything is usually first turned into triangles.
- One triangle is:
  - Three edges
  - Three vertices

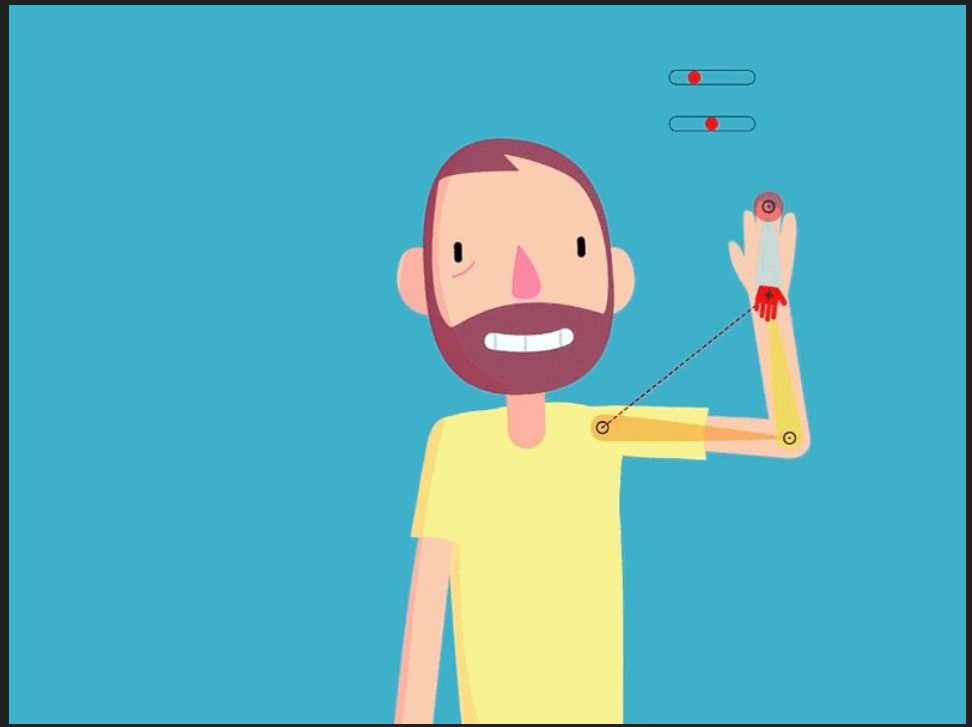


Sprites

- High-level representation of all the info that we need to render a texture in a 2D setting
  - Texture
  - Position and Size
  - Tint
  - Pivot

Sprite pivot

- The pivot is the “logical” center of a sprite.
- Defaults to the center of the sprite, but often useful to have it in other places
- Particularly useful for rotation
  Example: an arm should rotate around the joint, not its physical center.



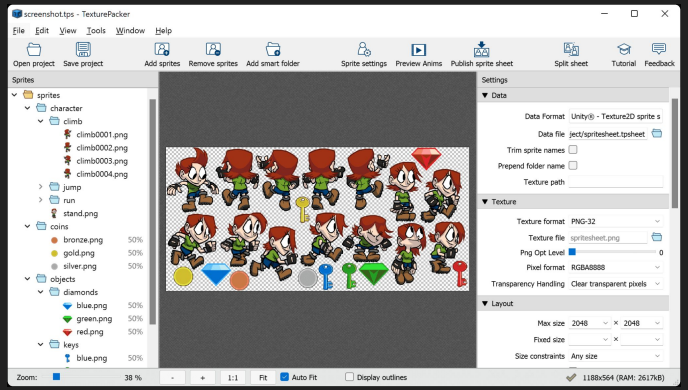
Sprites composition

Sprites can be rendered at runtime very efficient, and we can compose them in many different ways to give the illusion of virtual worlds.

Texture Atlases

- Usually it's much more efficient to pack multiple sprites in the same texture
- Arrangements of sprites is important:
  - arranged in a uniform grid
    - trivial to generate
    - trivial to get each sprite
    - more wasted space (especially with non-uniform sprites)
  - packed
    - requires dedicated software and algorithms to generate
    - extracting the sprite is more difficult
    - higher compression ratio

https://www.codeandweb.com/texturepacker



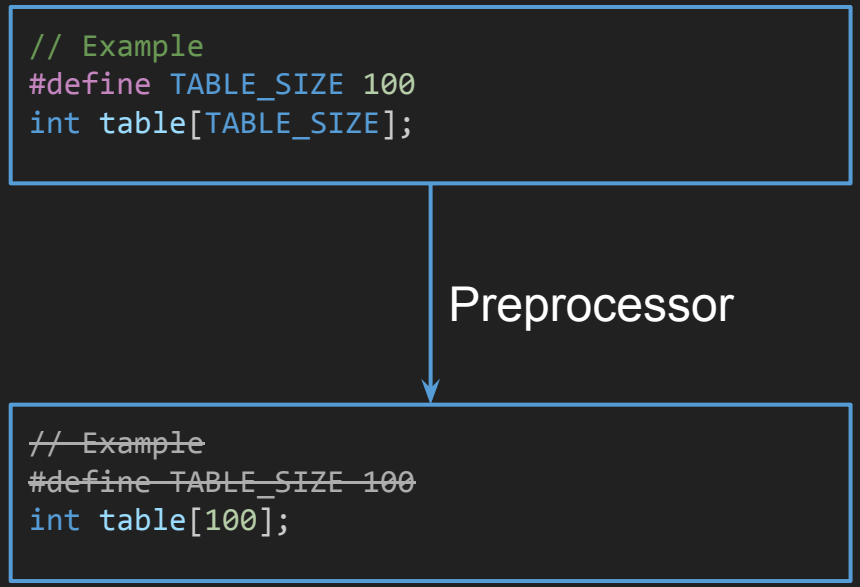
https://kenney.nl/assets

Agenda

1. Vector math recap
2. 2D Graphics
3. C++ Tidbits

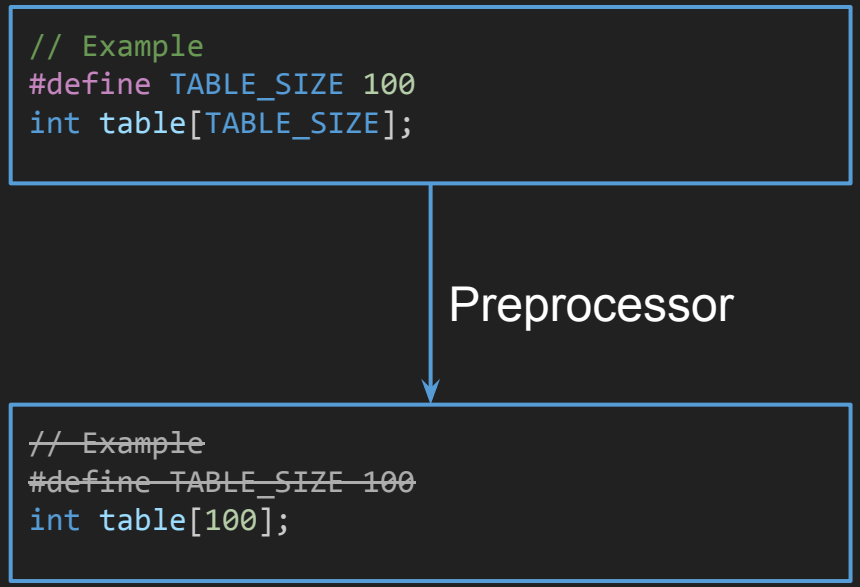
Preprocessor

- Text replacement function before compiling source
- Preprocessor directives starts with #
- No semicolon – Terminated with new-line
- Backslash (\) allows multiline



Preprocessor uses

- inject information in the program at compile time
- implements some degree of meta-programming
- allows to properly compile programs across multiple files
- true constant values (we'll talk about the `const` keyword another time…)



Preprocessor macros

- Creates a text-replacement macro
- Use with caution (if at all)
- You can create some really strange code

```
// Replaces F(1,2) with 1+2
#define F(a,b) a+b

void macro_quiz() {
   int x = 3;
   printf("a: %d\n",  F(x, 2)*2);
   printf("b: %d\n",  F(-,x));
}


// a: ???
// b: ???
```

Preprocessor macros - the bad

```c
#define SQUARE(x) ((x) * (x))

int x = 2;
printf("square: %d\n",  SQUARE(x++));

// square: 6???
```

Preprocessor macros - the bad

```c
#define SQUARE(x) ((x) * (x))


int x = 2;
printf("square: %d\n",  SQUARE(x++));


// square: 6???
```

```c
#include <stdio.h>
#define System S s;s
#define public
#define static
#define void int
#define main(x) main()
struct F{void println(char* s){puts(s);}};
struct S{F out;};

public static void main(String[] args) {
   System.out.println("Hello World!");
}
```

Preprocessor macros - the bad

```
#define SQUARE(x) ((x) * (x))

int x = 2;
printf("square: %d

// square: 6???
```

```
// I maintain code that has gotos in macros.
// So a function will have a label at the end
// but no visible goto in the function code.
// To make matters worse the macro is at the end
// of other statements usually off the screen
// unless you scroll horizontally.

#define CHECK_ERROR if (!true) goto Cleanup;

void SomeFunction()
{
   SomeLongFunctionName(ParamOne, ParamTwo, ParamThree, ParamFour); CHECK_ERROR
   //SomeOtherCode
   Cleanup:
   //Cleanup code
}
```

```
#include <stdio.h>
#define System S s
#define public
#define static
#define void int
#define main(x) ma
struct F{void prin
struct S{F out;};

public static void main(String[] args) {
   System.out.println("Hello World!");
}
```

Preprocessor macros - the bad

```
#define SQUARE(x) ((x) * (x))

int x = 2;
printf("square: %d

// square:
```

```
#define private public
```

```
// I maintai                has gotos in macros.
// So a                 ill have a label at the end
//                   ble goto in the function code.
//  matters worse the macro is at the end
//  other statements usually off the screen
//  unless you scroll horizontally.

#define CHECK_ERROR if (!true) goto Cleanup;

void SomeFunction()
{
    SomeLongFunctionName(ParamOne, ParamTwo, ParamThree, ParamFour); CHECK_ERROR
    //SomeOtherCode
    Cleanup:
    //Cleanup code
}
```

```
#include <stdio.h>
#define System S s
#define public
#define static
#define void int
#define main(x) ma
struct F{void prin
struct S{F out;};

public static void main(String[] args) {
   System.out.println("Hello World!");
}
```

Preprocessor macros - the bad

```
#define SQUARE(x) ((x) * (x))

int x = 2;
printf("square: %d

// square:
```

```
#define private public
```

```
// I maintai            has gotos in macros.
// So a                 ill have a label at the end
//                      ble goto in the function code.
//                      matters worse the macro is at the end
//                      other statements usually off the screen
// unless you scroll horizontally.

#define CHECK_ERROR if (!true) goto Cleanup;

void SomeFunction()
{
    SomeLongFunctionName(ParamOne, ParamTwo, ParamThree, ParamFour); CHECK_ERROR
    //SomeOtherCode
    Cleanup:
    //Cleanup code
}
```

```
#define begin {
#define end   }
```

```
#include <stdio.h>
#define System S s
#define public
#define static
#define void int
#define main(x) ma
struct F{void prin
struct S{F out;};

public static void main(String[] args) {
   System.out.println("Hello World!");
}
```

Preprocessor macros - the bad

```
#define SQUARE(x) ((x) * (x))
```

What is the worst real-world macros/pre-processor abuse you've ever come across?
https://stackoverflow.com/q/652788

```
#define main(x) ma
struct F{void prir
struct S{F out;};

public static void main(String[] args) {
   System.out.println("Hello World!");
}
```

```
    Cleanup:
    //Cleanup code
}
```

Macros - Useful Example

```
// mem.cpp
#include <stdlib.h>

#define MEM_CHUNK_SIZE 128

int main() {
   void* a = malloc(MEM_CHUNK_SIZE);
   free(a);
}
```

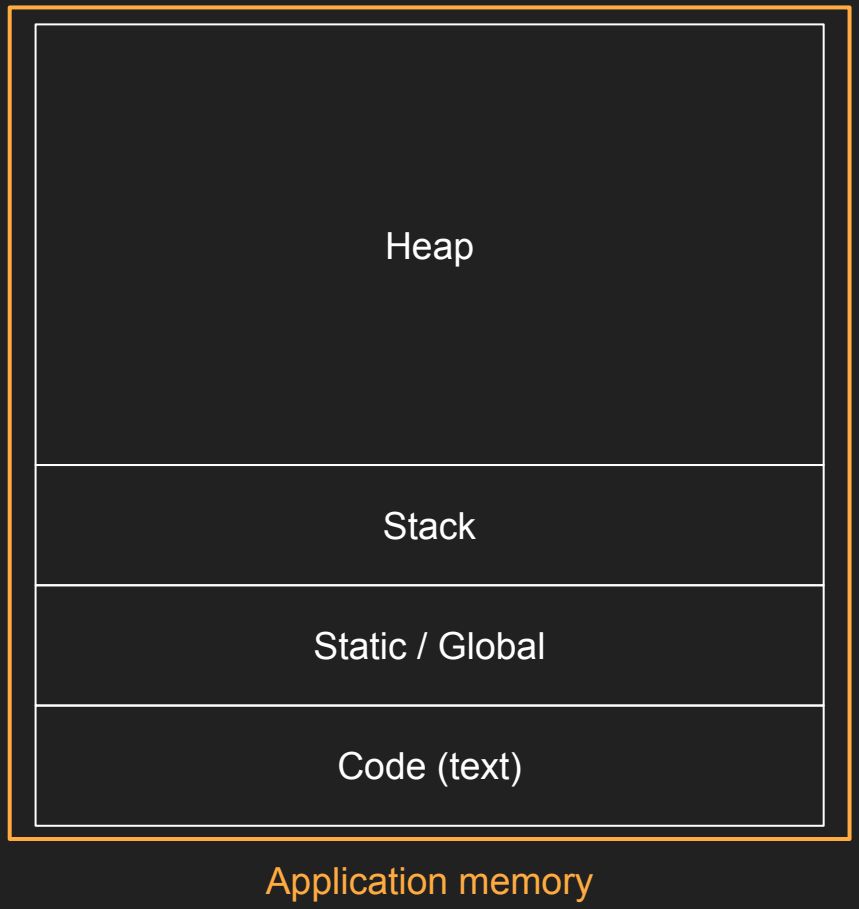
Macros - Useful Example

```
// mem.cpp
#include <mem_utils.h>

#define MEM_CHUNK_SIZE 128

int main() {
   void* a = malloc(MEM_CHUNK_SIZE);
   free(a);
}

output:
$ ./a.exe
main.cpp:39: alloc 128 bytes at addr. 295F5DA1F50
main.cpp:40: dealloc memory at addr 295F5DA1F50
```

```
// mem_utils.hpp
#include <stdio.h>
#include <stdlib.h>

#ifdef DEBUG_MEM_ALLOCATIONS

void* debug_malloc(size_t size, const char* file, int line) {
   void* mem = malloc(size);

   printf(
      "%s:%d: alloc %lld bytes at addr. %p\n",
      file, line, size, mem
   );

   return mem;
}

void debug_free(void* mem, const char* file, int line) {
   free(mem);
   printf(
      "%s:%d: dealloc memory at addr %p\n",
      file, line, mem
   );
}

#define malloc(x) debug_malloc(x, __FILE__, __LINE__)
#define free(x)   debug_free(x, __FILE__, __LINE__)

#endif
```

C/C++ Memory Layout

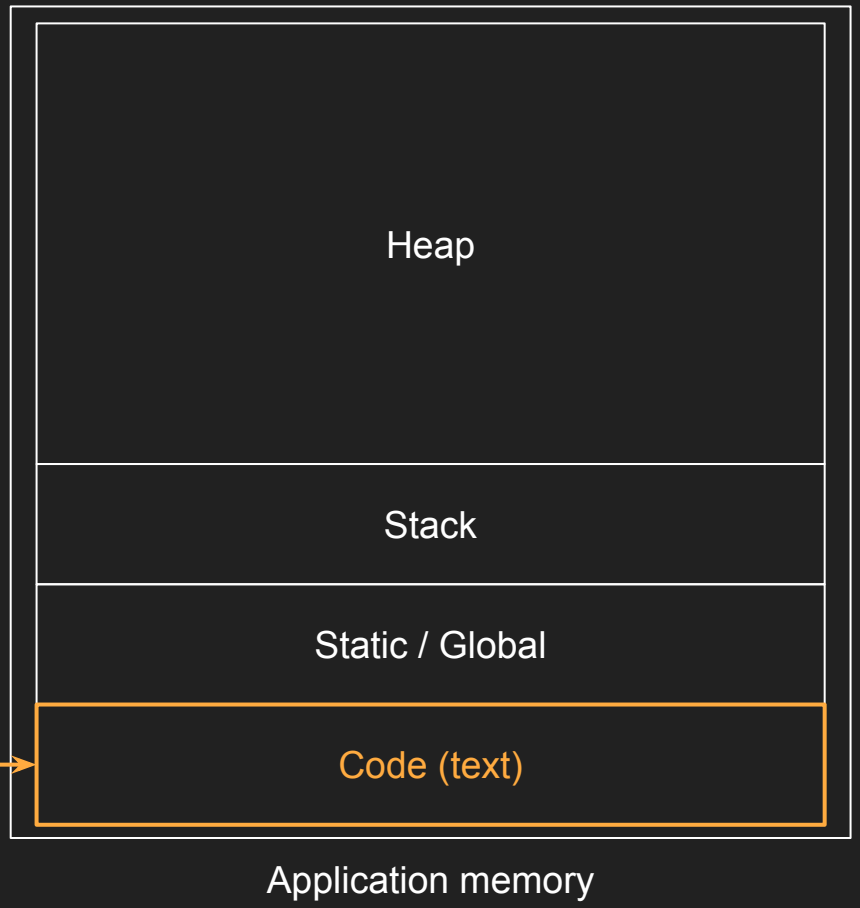
- Allocated on program startup by OS
- No access memory outside application (protected by OS)
- Using virtual memory addresses (different from physical memory addresses)
- Destroyed on program exit by OS



Code (Text) segment

- Contains the machine code for the program
- It's data like everything else, so we can reference it with variables! (function pointers)

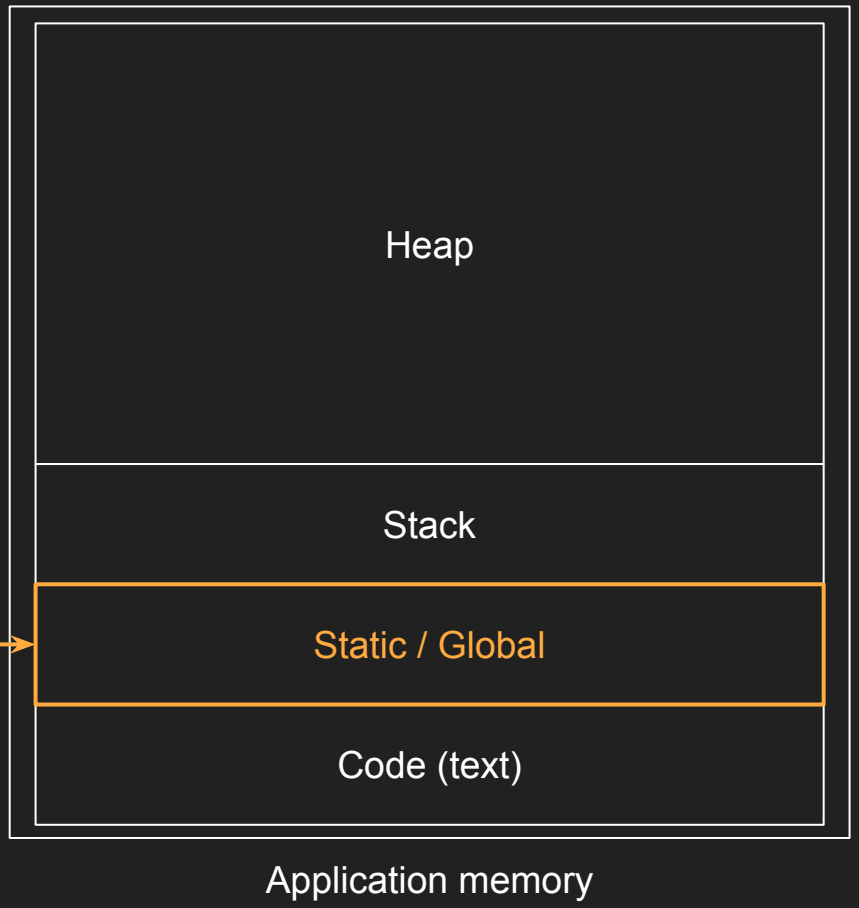
```
_start:
   mov edx,len
   mov ecx,msg
   mov ebx,1
   mov eax,4
   int 0x80
```



Static / Global segment

- Contains
  - global variables
  - static variables

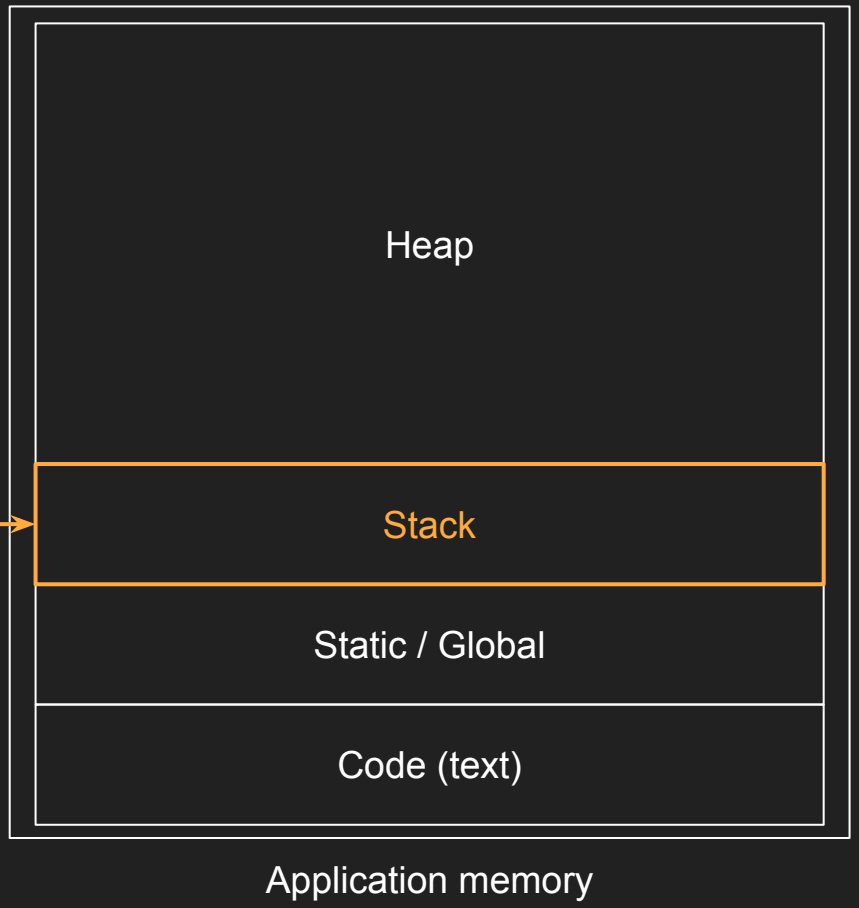
```
int x;
int foo(){
   static int y = 123;
   return y++;
}
```



Stack

- Contains
  - local variables
  - function parameters
- Has a fixed (and comparatively small) size
- Size of each stack frame is known at compile-time

```
int fibonacci (int x)
{
   if (x <= 1) return 1;
   return fibonacci (x-1) +
          fibonacci (x-2);
}
```



Heap

- General purpose memory
- Has a fixed (and comparatively small) size
- Dynamically allocated (anything created with alloc function goes here)
- Grows as needed

```
void entities_create(Entity* entities, int num) {
   entities = (Entity*)malloc(sizeof(Entity) * num);
}

void entitites_destroy(Entity* entities) {
   free(entities);
}
```